

JOBSHEET 14
LAPORAN PERCOBAAN PRAKTIKUM
TREE

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

1. Percobaan 1 : Implementasi Binary Search Tree Menggunakan Linked List

Mahasiswa14.java

```
package Pertemuan14;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14() {

    }

    Mahasiswa14(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampilInformasi() {
        System.out.println("NIM : " + this.nim + " " + "Nama : " + this.nama
            + " " + "Kelas : " + this.kelas + " " + "IPK : " + this.ipk);
    }
}
```

Node14.java

```
package Pertemuan14;

public class Node14 {
    Mahasiswa14 mahasiswa;
    Node14 left, right;

    Node14() {

    }

    Node14(Mahasiswa14 mahasiswa) {
        this.mahasiswa = mahasiswa;
    }
}
```

```

        left = right = null;
    }

}

```

BinaryTree14.java

```

package Pertemuan14;

public class BinaryTree14 {
    Node14 root;

    BinaryTree14() {
        root = null;
    }

    boolean isEmpty() {
        return (root == null);
    }

    void add(Mahasiswa14 mahasiswa) {
        Node14 newNode = new Node14(mahasiswa);

        if (isEmpty()) {
            root = newNode;
        }
        else {
            Node14 current = root;
            Node14 parent = null;

            while (true) {
                parent = current;

                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;

                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                }
                else {
                    current = current.right;

                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }
}

```

```

boolean find(double ipk) {
    boolean result = false;
    Node14 current = root;

    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        }
        else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        }
        else {
            current = current.left;
        }
    }

    return result;
}

void traversePreOrder(Node14 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(Node14 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(Node14 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

Node14 getSuccessor(Node14 del) {
    Node14 successor = del.right;
    Node14 successorParent = del;

    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }

    if (successor != del.right) {
        successorParent.left = successor.right;
    }
}

```

```

        successor.right = del.right;
    }

    return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }

    // cari node (current) yang akan dihapus
    Node14 parent = root;
    Node14 current = root;

    boolean isLeftChild = false;

    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        }
        else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        }
        else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // PENGHAPUSAN
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    }
    else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            }
            else {
                if (isLeftChild) {
                    parent.left = null;
                }
                else {
                    parent.right = null;
                }
            }
        }
        else if (current.left == null) { // jika hanya punya 1 anak (kanan)

```

```
public class BinaryTreeMain14 {
    public static void main(String[] args) {
```

```

BinaryTree14 bat = new BinaryTree14();

bat.add(new Mahasiswa14("244160121", "Ali", "A", 3.57));
bat.add(new Mahasiswa14("244160221", "Badar", "B", 3.85));
bat.add(new Mahasiswa14("244160185", "Candra", "C", 3.21));
bat.add(new Mahasiswa14("244160220", "Dewi", "B", 3.54));

System.out.println("\nDaftar semua mahasiswa (is order traversal) : ");
bat.traverseInOrder(bat.root);

System.out.println("\nPencarian data mahasiswa : ");
System.out.print("Cari mahasiswa dengan IPK : 3.54 : ");
String hasilCari = bat.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
System.out.println(hasilCari);

System.out.print("Cari mahasiswa dengan IPK : 3.22 : ");
hasilCari = bat.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
System.out.println(hasilCari);

bat.add(new Mahasiswa14("244160131", "Devi", "A", 3.72));
bat.add(new Mahasiswa14("244160205", "Ehsan", "D", 3.37));
bat.add(new Mahasiswa14("244160170", "Fizi", "B", 3.46));

System.out.println("\nDaftar semua mahasiswa setelah penambahan 3 mahasiswa : ");
System.out.println("InOrder Traversal : ");
bat.traverseInOrder(bat.root);
System.out.println("PreOrder Traversal : ");
bat.traversePreOrder(bat.root);
System.out.println("PostOrder Traversal : ");
bat.traversePostOrder(bat.root);

System.out.println("\nPenghapusan data mahasiswa");
bat.delete(3.57);
System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in
order traversal)");
bat.traverseInOrder(bat.root);

}
}

```

Verifikasi Hasil Percobaan

```
Daftar semua mahasiswa (is order traversal) :
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85

Pencarian data mahasiswa :
Cari mahasiswa dengan IPK : 3.54 : Ditemukan
Cari mahasiswa dengan IPK : 3.23 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa :
InOrder Traversal :
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.37
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.46
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85

PreOrder Traversal :
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.37
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.46
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72

PostOrder Traversal :
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.46
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.37
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
NIM : 244160121 Nama : Ali Kelas : A IPK : 3.57

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal)
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.37
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.46
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
```

Pertanyaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?
2. Untuk apakah di class **Node**, kegunaan dari atribut **left** dan **right**?
3. a. Untuk apakah kegunaan dari atribut **root** di dalam class **BinaryTree**?
b. Ketika objek tree pertama kali dibuat, apakah nilai dari **root**?
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?
5. Perhatikan method **add()**, di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detail untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

6. Jelaskan langkah-langkah pada method **delete()** saat menghapus sebuah node yang memiliki dua anak. Bagaimana method **getSuccessor()** membantu dalam proses ini?

Jawaban

1. Proses pencarian data dalam Binary Search Tree (BST) jauh lebih efektif dibanding Binary Tree biasa karena adanya aturan penempatan data yang terstruktur. Pada BST, cabang sebelah kiri selalu berisi data yang lebih kecil dari root, sedangkan cabang sebelah kanan selalu berisi data yang lebih besar.

Aturan ini membuat komputer bisa menggunakan prinsip *divide and conquer*, yaitu langsung mengeliminasi atau membuang setengah jalur data di setiap tingkatan node yang dikunjungi tanpa harus memeriksa seluruh isi pohon satu per satu dari atas sampai bawah.

Proses pemangkasan jalur yang konstan ini membuat waktu pencarian berjalan sangat cepat dalam skala logaritmik ($\log n$), di mana tree dengan 1.000 data hanya membutuhkan maksimal sekitar 10 kali pengecekan untuk menemukan data yang dicari.

2. Sebagai pointer/referensi yang menunjuk ke node child dari node tersebut. Atribut **left** untuk menunjuk ke child di sebelah kiri (yang bernilai lebih kecil dari current node) dan atribut **right** untuk menunjuk ke child di sebelah kanan (yang bernilai lebih besar dari current node).
3. A. Sebagai wadah objek untuk menyimpan nilai root dari sebuah tree

B. Nilai null saat pertama objek tree dibuat adalah null, sebagai tanda bahwa tree masih kosong.

4. Ketika tree kosong, dan ditambahkan node baru, maka akan terjadi proses pemberian nilai pada objek root dengan object newNode yang mana merupakan node baru yang ditambahkan tadi.
5. Potongan kode tersebut artinya adalah sebagai berikut.

Dalam perulangan while true, program menginisialisasikan parent dengan nilai current. Kemudian program mengecek apakah ipk mahasiswa yang baru kurang dari ipk mahasiswa yang sedang dicek saat ini. Jika ya, maka pointer bergerak ke kiri karena letak nilai yang kecil ada di sebelah kiri. Kemudian program mengecek lagi apakah setelah perubahan pointer tadi, pointer bernilai null. Jika ya, artinya node sebelumnya adalah leaf dan merupakan posisi terbaik untuk ipk mahasiswa yang baru, sehingga program akan mengisikan nilai parent.left dengan node baru tadi.

Namun, jika tidak, jika ipk mahasiswa yang baru lebih dari ipk mahasiswa yang sedang dicek saat ini, maka kebalikannya, pointer bergerak ke kanan karena letak nilai yang lebih besar ada di sebelah kanan. Kemudian seperti sebelumnya, program mengecek apakah setelah perubahan pointer tadi, pointer bernilai null. Jika ya, artinya node sebelumnya adalah leaf dan merupakan posisi terbaik untuk ipk mahasiswa yang baru, sehingga program akan mengisikan nilai parent.right dengan node baru tadi.

6. Konsep penghapusan node yang memiliki 2 anak adalah mencari successor dari node yang akan dihapus untuk dijadikan pengganti node tadi.

Saat menghapus node yang memiliki dua anak, method delete() tidak bisa langsung menghapus node tersebut karena akan memutuskan hubungan seluruh cabang di bawahnya. Untuk mengatasinya, method delete() meminta bantuan method getSuccessor() untuk mencari satu node pengganti yang valid, yaitu node dengan nilai terkecil di cabang sebelah kanan (del.right). Method getSuccessor() bekerja dengan cara melompat ke cabang kanan node yang akan dihapus, lalu menelusuri cabang kiri terus-menerus hingga mentok di ujung paling bawah untuk menemukan node pengganti tersebut.

Setelah node pengganti ditemukan, method getSuccessor() akan mengamankan struktur pohon dengan memindahkan anak kanan milik node pengganti ke sisi kiri orang tuanya (jika ada), lalu mengangkat node pengganti tersebut untuk mengambil alih kendali seluruh cabang kanan milik node yang akan dihapus. Terakhir, method delete() tinggal menerima node pengganti yang sudah ditata ini, menghubungkannya ke cabang kiri dari node lama yang dihapus, dan menyambungkannya ke parent utama. Dengan begitu, node lama berhasil terhapus dan struktur urutan Binary Search Tree tetap terjaga dengan rapi.

2. Percobaan 2 : Implementasi Binary Tree dengan Array

BinaryTreeArray14.java

```
package Pertemuan14;

public class BinaryTreeArray14 {
    Mahasiswa14[] dataMahasiswa;
    int idxLast;

    BinaryTreeArray14() {
        this.dataMahasiswa = new Mahasiswa14[10];
    }

    void populateData(Mahasiswa14 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

BinaryTreeArrayMain14.java

```
package Pertemuan14;

public class BinaryTreeArrayMain14 {
    public static void main(String[] args) {
        BinaryTreeArray14 bta = new BinaryTreeArray14();

        Mahasiswa14 mhs1 = new Mahasiswa14("244160131", "Devi", "A", 3.72);
        Mahasiswa14 mhs2 = new Mahasiswa14("244160221", "Badar", "B", 3.85);
        Mahasiswa14 mhs3 = new Mahasiswa14("244160185", "Candra", "C", 3.21);
        Mahasiswa14 mhs4 = new Mahasiswa14("244160220", "Dewi", "B", 3.54);

        Mahasiswa14 mhs5 = new Mahasiswa14("244160131", "Devi", "A", 3.72);
        Mahasiswa14 mhs6 = new Mahasiswa14("244160205", "Ehsan", "D", 3.37);
        Mahasiswa14 mhs7 = new Mahasiswa14("244160170", "Fizi", "B", 3.46);

        Mahasiswa14[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7, null,
        null, null};
        int idxLast = 6;

        bta.populateData(dataMahasiswas, idxLast);
        System.out.println("\nInOrder Traversal Mahasiswa : ");
    }
}
```

```

        bta.traverseInOrder(0);
    }
}

```

Verifikasi Hasil Percobaan

```

InOrder Traversal Mahasiswa :
NIM : 244160220 Nama : Dewi Kelas : B IPK : 3.54
NIM : 244160221 Nama : Badar Kelas : B IPK : 3.85
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160131 Nama : Devi Kelas : A IPK : 3.72
NIM : 244160205 Nama : Ehsan Kelas : D IPK : 3.37
NIM : 244160185 Nama : Candra Kelas : C IPK : 3.21
NIM : 244160170 Nama : Fizi Kelas : B IPK : 3.46

```

Pertanyaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class **BinaryTreeArray**?
2. Apakah kegunaan dari method **populateData()**?
3. Apakah kegunaan dari method **traverseInOrder()**?
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?
6. Mengapa indeks `2*idxStart+1` dan `2*idxStart+2` digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawaban

1. Atribut data berfungsi sebagai wadah atau tempat penyimpanan objek/nilai dari node-node pohon yang posisinya ditentukan berdasarkan rumus indeks berbasis array. Sementara itu, atribut idxLast berfungsi sebagai penanda atau batas indeks terakhir yang sudah terisi di dalam array tersebut, sehingga sistem dapat mengetahui batas maksimal data pohon saat melakukan proses penelusuran (traversal) dan memastikan program tidak membaca indeks kosong yang melampaui kapasitas data yang ada.
2. Method `populateData()` pada `BinaryTreeArray` digunakan untuk memasukkan data awal ke dalam array pohon sekaligus menentukan batas akhir data tersebut.
3. Method ini digunakan untuk menampilkan semua data yang ada dalam tree array dengan urutan yang rapi, yakni kecil ke besar.
4. Menggunakan rumus pada teori ASD, untuk mencari left child adalah dengan mengoperasikan $2 * i + 1 = 2 * 2 + 1 = 5$. Kemudian untuk right child adalah dengan mengoperasikan $2 * i + 2 = 2 * 2 + 2 = 6$.
5. Untuk menandai atau menetapkan bahwa batas akhir elemen (node) yang memiliki nilai/data di dalam array pohon tersebut berada di indeks ke-6.

6. Rumus tersebut adalah formula matematika baku untuk memetakan hubungan orang tua (parent) dan anak (child) di dalam array. Karena tidak ada pointer fisik, posisi node diatur lewat perhitungan indeks. Kaitannya dengan binary tree adalah sebagai berikut :

- $2 * \text{idxStart} + 1$ selalu digunakan untuk melompat ke posisi indeks anak kiri
- $2 * \text{idxStart} + 2$ selalu digunakan untuk melompat ke posisi indeks anak kanan.

Saat dipanggil secara rekursif, rumus ini otomatis mengubah posisi indeks idxStart menjadi lebih besar, yang berarti komputer sedang bergerak turun ke level bawah pohon untuk memeriksa cabang-cabangnya hingga mencapai batas akhir data.